

Error while getting meta information about the page.

[Home](#) [Whiteboard](#) [Graphing Calculator](#) [Online Compilers](#) [Articles](#) [Tools](#)

[Home](#) > [Python](#) >

Packing and Unpacking in Python

Packing and unpacking are a techniques used in Python to handle multiple values in a single variable. This is similar to how we pack items into a bag and unpack them when needed. Python functions can be used to accept a packed set of arguments, which can then be unpacked inside the function for further processing. This is called as **packing** and **unpacking** of **arguments**.

In this chapter, we will discuss all the details of packing and unpacking in Python and how it is performed inside function arguments. Following are the topics that we will cover in this chapter –

- [What is Packing and Unpacking?](#)
- [Packing and Unpacking Tuples in Python](#)
- [Packing Arguments in Python](#)
- [Unpacking Arguments in Python](#)
- [Use Case of Packing and Unpacking Arguments](#)
- [Comparison of Packing and Unpacking Arguments](#)

What is Packing and Unpacking?

Packing is a technique used to group multiple values into a single variable. For example, while calling a function, you can pass multiple arguments by packing them into a tuple or list. **Unpacking** is just the reverse process of packing. It is used to extract individual values from a packed variable and assign them to separate variables. Generally, an unpacking is performed inside any functions to access the individual values from a packed variable.

In Python, packing and unpacking can be done using the following symbols –

- * – This is used to pack or unpack a variable number of positional arguments.
- ** – This is used to pack or unpack a variable number of keyword arguments.

Let's look at an **example** to understand how packing and unpacking works in Python –

```
# Packing Example
def pack_example(*args):
    print("Packed arguments:", args)
pack_example(1, 2, 3, 'a', 'b')

# Unpacking Example
def unpack_example(a, b, c):
    print("Unpacked arguments:", a, b, c)
values = [1, 2, 3]
unpack_example(*values)
```

The **output** of the above code will be –

```
Packed arguments: (1, 2, 3, 'a', 'b')
Unpacked arguments: 1 2 3
```

Packing and Unpacking Tuples in Python

In the context of tuples, packing refers to the process of creating a tuple by grouping multiple values together. Similarly, unpacking refers to the process of extracting individual values from a tuple and assigning them to separate variables. This operation is useful for functions that return multiple values as a tuple.

You can use following **syntax** to pack and unpack tuples in Python –

```
# Packing a tuple
my_tuple = (1, 2, 3)

# Unpacking a tuple
a, b, c = my_tuple
```

Example 1

In this example, the tuple contains 6 values and variables to be unpacked are 3. We prefix "*" to the second variable.

```
tup1 = (10,20,30, 40, 50, 60)
x, *y, z = tup1
print ("x: ",x, "y: ", y, "z: ", z)
```

It will produce the following **output** –

```
x: 10 y: [20, 30, 40, 50] z: 60
```

Here, the values are unpacked in "x" and "z" first, and then the rest of the values are assigned to "y" as a list.

Example 2

In this example, we will add the prefix "*" to the first variable and see what will be the output.

```
tup1 = (10,20,30, 40, 50, 60)
*x, y, z = tup1
print ("x: ",x, "y: ", y, "z: ", z)
```

It will produce the following **output** –

```
x: [10, 20, 30, 40] y: 50 z: 60
```

Here again, the tuple is unpacked in such a way that individual variables take up a single value first, leaving the remaining values to the list "x".

Packing Arguments in Python

The packing arguments refer to the process of collecting multiple arguments into a single function parameter. This is useful when you pass an iterable (like a list or tuple) or a dictionary to a function and want to combine them into a single parameter.

There are **two ways** to pack arguments in Python -

- ***args**: This is used to pack a multiple number of **positional arguments** into a tuple.
- ****kwargs**: This is used to pack a multiple number of **keyword arguments** into a dictionary.

Packing Arguments with *args

In the following **example**, we declared a function that uses ***args** to combine multiple positional arguments into a single tuple. We then print the whole tuple using a single print statement.

```
def pack_args(*args):  
    print("Packed arguments:", args)  
  
# Calling the function with multiple arguments  
pack_args(1, 2, 3, 'a', 'b')
```

The **output** of the above code will be –

```
Packed arguments: (1, 2, 3, 'a', 'b')
```

Packing Arguments with **kwargs

In the **example** below, we declared a function that uses ****kwargs** to combine multiple keyword arguments into a single dictionary. The function then prints the whole dictionary.

```
def pack_kwargs(**kwargs):  
    print("Packed keyword arguments:", kwargs)  
  
# Calling the function with multiple keyword arguments  
pack_kwargs(name="Alice", age=30, city="New York")
```

The **output** of the above code will be –

```
Packed keyword arguments: {'name': 'Alice', 'age': 30, 'city': 'New York'}
```

Unpacking Arguments in Python

The **unpacking arguments** is just the opposite of packing arguments. In this case, we will pass an entire collection (like a list, tuple, or dictionary) to a function and unpack its elements into individual arguments inside the function. There are two ways to unpack arguments in Python.

Unpacking Arguments with *

The *** operator** is used to unpack a list or tuple into individual positional arguments. In the **example** shown below, we declared a function that takes three positional arguments. We then unpack a list of values into individual arguments when calling the function.

```
def unpack_args(a, b, c):  
    print("Unpacked arguments:", a, b, c)  
  
# List of values to unpack  
values = [1, 2, 3]  
# Calling the function with unpacked values  
unpack_args(*values)
```

The **output** of the above code will be –

```
Unpacked arguments: 1 2 3
```

Unpacking Arguments with **

The **** operator** is used to unpack a dictionary into individual keyword arguments. In the following **example**, we declared a function that takes three keyword arguments. We then unpack a dictionary of values into individual keyword arguments when calling the function.

```
def unpack_kwargs(name, age, city):
    print("Unpacked keyword arguments:", name, age, city)

# Dictionary of values to unpack
info = {"name": "Farhan", "age": 25, "city": "Los Angeles"}

# Calling the function with unpacked values
unpack_kwargs(**info)
```

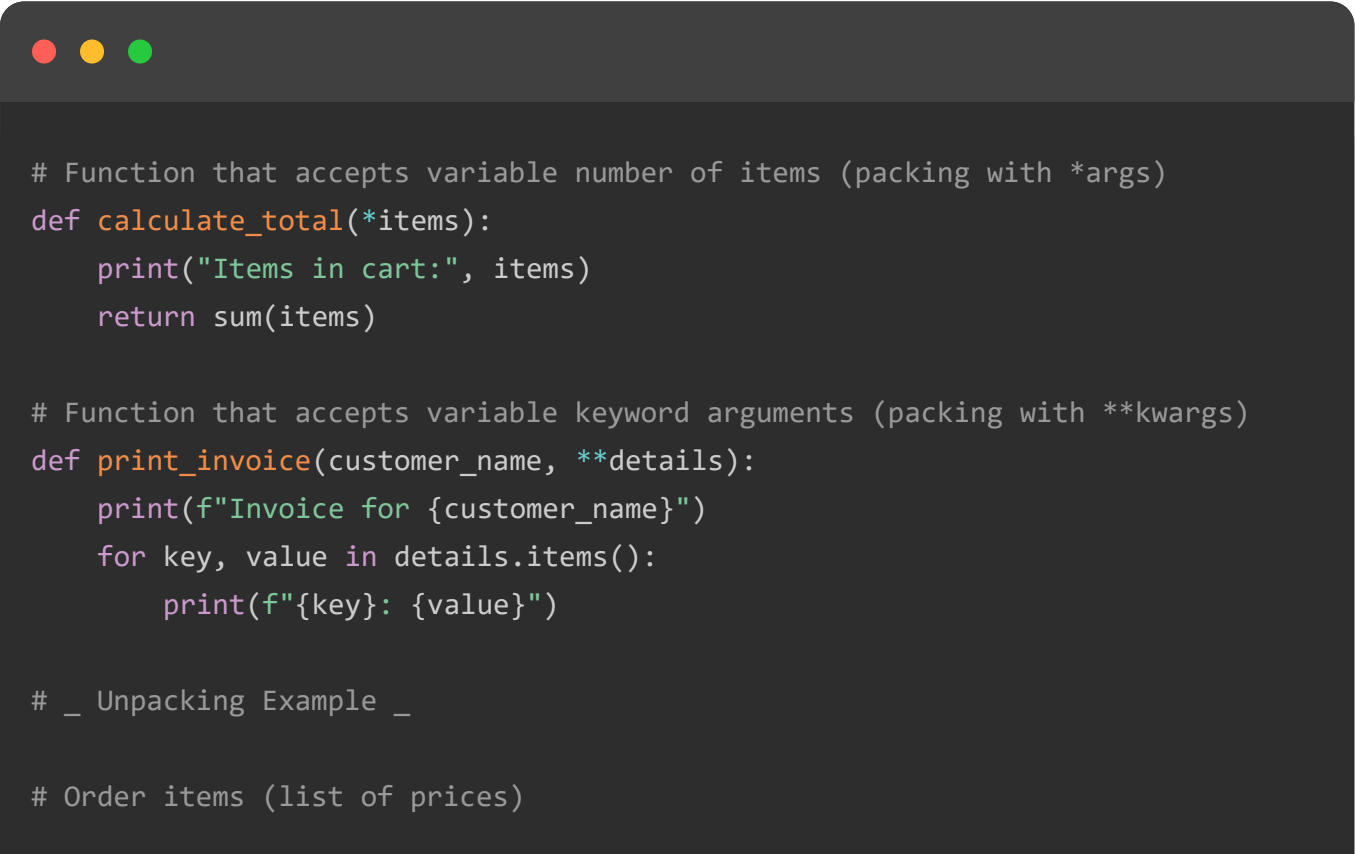
The **output** of the above code will be –

```
Unpacked keyword arguments: Farhan 25 Los Angeles
```

Real-World Use-Case of Packing and Unpacking Arguments

Packing and unpacking arguments are commonly used in real-world applications. Here is an **example of a shopping cart application** where we use packing and unpacking arguments to handle variable numbers of items and their details.

In this **example**, we have two functions: one for calculating the total price of items in the cart using packing with ***args**, and one for printing the invoice details using packing with ****kwargs**. We also show unpacking a list of item prices and a dictionary of item details when calling these functions.



```
# Function that accepts variable number of items (packing with *args)
def calculate_total(*items):
    print("Items in cart:", items)
    return sum(items)

# Function that accepts variable keyword arguments (packing with **kwargs)
def print_invoice(customer_name, **details):
    print(f"Invoice for {customer_name}")
    for key, value in details.items():
        print(f"{key}: {value}")

# _ Unpacking Example _

# Order items (list of prices)
```

```

order_items = [250, 100, 75] # Prices of products
total_amount = calculate_total(*order_items) # Unpacking list
print("Total Amount:", total_amount)

# Order details (dictionary)
order_details = {
    "Product": "Laptop",
    "Quantity": 1,
    "Price": total_amount,
    "Payment Method": "Credit Card"
}

# Passing dictionary as keyword args
print_invoice("Farhan", **order_details)

```

The **output** of the above code will be –

```

Items in cart: (250, 100, 75)
Total Amount: 425
Invoice for Farhan
Product: Laptop
Quantity: 1
Price: 425
Payment Method: Credit Card

```

Comparison of Packing and Unpacking Arguments

Here is a detailed comparison of packing and unpacking arguments in Python –

Feature	Packing Arguments	Unpacking Arguments
Definition	Packing arguments is the process of collecting multiple arguments into a single parameter using *args or **kwargs .	Unpacking arguments is the process of breaking down a collection (like a list, tuple, or dictionary) into individual arguments using * or ** .
Purpose	Allows functions to accept a variable number of arguments.	Allows passing values dynamically to functions or variables.

Working	Combine multiple values into a single entity (tuple or dictionary).	Break down a single entity (list, tuple, or dictionary) into multiple values.
Use Case	Commonly used when the number of arguments is unknown or variable.	Commonly used when you have a collection of values that need to be passed as individual arguments.
Example Syntax	<code>def func(*args, **kwargs):</code>	<code>func(*list_of_values)</code> or <code>func(**dict_of_values)</code>

Conclusion

Packing and unpacking arguments are new features in Python that allow us to write flexible function definitions and calls. One important thing to note is that when using both ***args** and ****kwargs** in a function definition, ***args** must appear before ****kwargs**. This ensures that positional arguments are collected first, then the keyword arguments.

TOP TUTORIALS

[Python Tutorial](#)
[Java Tutorial](#)
[C++ Tutorial](#)
[C Programming Tutorial](#)
[C# Tutorial](#)
[PHP Tutorial](#)
[R Tutorial](#)
[HTML Tutorial](#)
[CSS Tutorial](#)
[JavaScript Tutorial](#)
[SQL Tutorial](#)

TRENDING TECHNOLOGIES

[Cloud Computing Tutorial](#)
[Amazon Web Services Tutorial](#)
[Microsoft Azure Tutorial](#)
[Git Tutorial](#)

[Ethical Hacking Tutorial](#)

[Docker Tutorial](#)

[Kubernetes Tutorial](#)

[DSA Tutorial](#)

[Spring Boot Tutorial](#)

[SDLC Tutorial](#)

[Unix Tutorial](#)

CERTIFICATIONS

[Business Analytics Certification](#)

[Java & Spring Boot Advanced Certification](#)

[Data Science Advanced Certification](#)

[Cloud Computing And DevOps](#)

[Advanced Certification In Business Analytics](#)

[Artificial Intelligence And Machine Learning](#)

[DevOps Certification](#)

[Game Development Certification](#)

[Front-End Developer Certification](#)

[AWS Certification Training](#)

[Python Programming Certification](#)

COMPILERS & EDITORS

[Online Java Compiler](#)

[Online Python Compiler](#)

[Online Go Compiler](#)

[Online C Compiler](#)

[Online C++ Compiler](#)

[Online C# Compiler](#)

[Online PHP Compiler](#)



Chapters ▾

Categories ≡

[Online SQL Compiler](#)

[Online Html Editor](#)

[ABOUT US](#) | [OUR TEAM](#) | [CAREERS](#) | [JOBS](#) | [CONTACT US](#) | [TERMS OF USE](#) |

[PRIVACY POLICY](#) | [REFUND POLICY](#) | [COOKIES POLICY](#) | [FAQ'S](#)



Tutorials Point is a leading Ed Tech company striving to provide the best learning material on technical and non-technical subjects.

© Copyright 2026. All Rights Reserved.